

Hecho por David Eduardo Rivera Ponce

He realizado este documento para responder a las siguientes preguntas:

1. ¿Cómo funciona el algoritmo?
2. ¿Cómo se explica la salida producida por el algoritmo?

El proyecto trabaja con diferentes algoritmos para **crear un compilador** que tiene la finalidad de **revisar si un programa** escrito en el archivo **programa.txt**, es **léxica y sintácticamente** correcto, siguiendo las reglas que son impuestas por la **gramática**.

Ahora, tenemos que analizar la gramática del proyecto:

<u>GRAMÁTICA LC</u>	
1.- programa	→ class id { lista_sent }
2.- lista_sent	→ sentencia sent_final
3.- sent_final	→ sentencia sent_final
4.- sent_final	→ ϵ
5.- sentencia	→ tipo lista_id ;
6.- sentencia	→ id = expresion ;
7.- sentencia	→ read (lista_id);
8.- sentencia	→ write (lista_expr);
9.- lista_id	→ id id_final
10.- id_final	→ , id id_final
11.- id_final	→ ϵ
12.- lista_expr	→ expresion lista_exprfinal
13.- lista_exprfinal	→ , expresion lista_exprfinal
14.- lista_exprfinal	→ ϵ
15.- expresion	→ expr_arit expr_final
16.- expr_final	→ operador expr_arit expr_final
17.- expr_final	→ ϵ
18.- expr_arit	→ (expresion)
19.- expr_arit	→ id
20.- expr_arit	→ enteros
21.- expr_arit	→ reales
22.- tipo	→ int
23.- tipo	→ float
24.- operador	→ +
25.- operador	→ -
26.- operador	→ *
27.- operador	→ /
 28.- inicio	→ programa

La gramática empieza a ejecutarse siguiendo el orden desde la línea:

28.- inicio la cual produce -> programa

Posteriormente **programa** produce **class id { lista_sent }** en la línea 1.-

Explicación:

Cada elemento del **lado izquierdo** de la flecha → en la gramática **produce** una acción que se encuentra **del lado derecho** de la flecha →

Del lado derecho de la flecha →, los **elementos en rojo NO producen una acción**, estos son elementos **finales irremplazables**.

En cambio, los **elementos en color negro** del lado derecho de la flecha → **SÍ** producen más acciones, estas acciones pueden derivar una cadena de más acciones aún.

Por ejemplo:

2.- lista_sent produce sentencia y sent_final

pero a su vez,

sentencia produce cualquiera de las siguientes acciones mostradas en la imagen siguiente: 5, 6, 7 u 8. La decisión de qué acción a producir (es decir, solo puede ejecutarse una acción, **NO** las 4), será tomada de acorde al programa escrito en **programa.txt** que se está analizando, y con la ayuda de la **matriz predictiva**.

5.- sentencia	→ tipo lista_id;
6.- sentencia	→ id = expresion ;
7.- sentencia	→ read (lista_id);
8.- sentencia	→ write (lista_expr);

Matriz predictiva (utilizada en el algoritmo):

No Term.	class	Id	{	}	;	=	read	(	)	write	,	enteros	reales	int	float	+	-	*	/
programa	1																		
lista_sent		2					2			2				2	2				
sent_final		3					3			3				3	3				
			4																
sentencia		6					7			8				5	5				
lista_id		9																	
Id_final					11				11		10								
lista_expr		12						12				12	12						
lista_expr final									14		13								
expresion		15						15				15	15						
expr_final									17		17					16	16	16	16
expr_arit		19						18				20	21						
tipo														22	23				
operador																24	25	26	27
inicio	28																		

Como se puede observar con lo anterior, cada acción (lo que aparece del lado derecho de → en color negro), puede desencadenar más acciones subsecuentes, para ello, se utiliza la estructura de datos 'Pila' para tener y seguir un orden de qué acción será la siguiente a analizar.

Ahora quedará más claro observando la salida (resultado) de nuestro programa:

El programa/código a analizar será:

```
≡ programa.txt X
src > archivos > ≡ programa.txt
1  class ejemplo {
2  int var1;
3  }
```

Al ejecutar el programa, el resultado obtenido en la consola es el siguiente:

1. **X** nos mostrará el **orden a seguir**, impuesto por la **gramática**.
2. **A** nos mostrará qué **Item** del **código** escrito en **programa.txt** está siendo analizado.
3. También tenemos el contenido actual de la **estructura de datos 'Pila'**, que nos ayudará a **seguir un orden de ejecución correcto** a lo largo de todo el programa.
4. Tenemos el resultado de **qué lado izquierdo produce qué lado derecho** en la **gramática**.
5. Tenemos una **tabla de símbolos**, la cual es muy útil para ayudarnos visualmente cómo va haciendo **match** cada **item** del **programa.txt** con los **elementos irremplazables** de la gramática (los de color rojo del lado derecho de →).

```
Corrida paso a paso del análisis Léxico-Sintáctico:
-----
Símbolo actual en X: inicio
Token actual en A: class
-----
Contenido actual de la pila:
inicio
-----
Producción Actual de Derivación:
28.-inicio ? programa $
-----
Tabla de símbolos
-----
LEXEMA      ATRIBUTO      CATEGORIA LÉXICA
-----
```

inicio produce **programa \$**, el análisis **debe tener un fin** y eso será indicado por el símbolo de pesos \$.

A partir de ahora, en lugar de decir **produce**, utilizaré una flecha →

La pila va acumulando **pasos/acciones** que tendrán que ser ejecutados posteriormente.

programa → class id { lista_sent }

```
-----
Símbolo actual en X: programa
Token actual en A: class
-----
Contenido actual de la pila:
programa
$
-----
Producción Actual de Derivación:
1.-programa ? class id { lista_sent }
-----
Tabla de símbolos
-----
LEXEMA      ATRIBUTO      CATEGORIA LÉXICA
-----
```

Ahora, lo que el algoritmo realiza es:

Se da cuenta que:

programa → class id { lista_sent }

Por lo que, **programa** YA cumplió con su deber, **se saca** de la **pila**, y ahora es turno de **ejecutar** la **acción correspondiente**:

→ class id { lista_sent }

Como se puede observar con la siguiente imagen, primero **se debe analizar** que el **programa.txt** comience con la palabra reservada '**class**', por lo cual, **está hasta arriba** en la '**Pila**', y a su vez '**}**' es el **último elemento a comprobar**, por lo que **programa.txt** tendrá que **finalizar con una llave de cierre '**

Al tener **X = A**, '**class**' hace match y el **programa.txt** va siendo **correcto**, de acuerdo a lo impuesto por la gramática:

```
-----
Símbolo actual en X: class
Token actual en A: class
-----
Contenido actual de la pila:
class
id
{
lista_sent
}
$
-----
Tabla de símbolos
-----
LEXEMA      ATRIBUTO      CATEGORIA LÉXICA
-----
```

```
-----
Símbolo actual en X: id
Token actual en A: ejemplo
-----
Contenido actual de la pila:
id
{
lista_sent
}
$
-----

Tabla de símbolos
-----
LEXEMA      ATRIBUTO      CATEGORIA LÉXICA
ejemplo      261      Identificadores
-----
```

Ahora, lo que **nos indica la pila** es que a continuación, **debemos tener un 'id'**, que en este caso **'id'** es el **nombre de la clase** ('ejemplo') del código escrito en **programa.txt**

¿Pero cómo sabemos que lo siguiente ('ejemplo') **será un 'id'**?

Lo haremos con la **Tabla de símbolos**.

Para ello con la ayuda de otro algoritmo implementado en el proyecto en el archivo **AnalizadorLexico.java**, identificamos que el ítem **'ejemplo'** tiene características de un **identificador** ('id'), es decir:

1. No es igual a una **palabra reservada del lenguaje** (como **class, int, float, etc**):

```
private String[][] palabrasReservadas = {
    //- Esta tabla deberá concordar con el lenguaje en estudio
    //- Los pares(lexema, atributo) de abajo son puramente un
    //- ejemplo.
    {"class", "262"},
    {"read", "263"},
    {"write", "264"},
    {"int", "265"},
    {"float", "266"}
};
```

2. No **comienza con un número** (lo cual produciría un error):

Error producido si se ejecuta **'1ejemplo'** (item comenzando con número), lo cual es un **'identificador'** incorrecto:

Incorrecto:

```
programa.txt X
src > archivos > programa.txt
1  class 1ejemplo {
2  int var1;
3  }
```

El compilador indicará error:

```
Windows PowerShell
-----
Símbolo actual en X: class
Token actual en A: class
-----
Contenido actual de la pila:
class
id
{
  lista_sent
}
$
-----
Tabla de símbolos
-----
LEXEMA      ATRIBUTO      CATEGORIA LÉXICA
-----
ERROR LEXICO: 1ejemplo en la línea 1
PS D:\TRASPASO\SEMESTER 5\Lenguajes Y Automatas\PROYECTO FINAL\AnalizadorLexicoSintactico>
```

Como nosotros estamos ejecutando el programa siguiente, con un **'identificador'** válido, podemos continuar sin errores:

```
programa.txt X
src > archivos > programa.txt
1  class ejemplo {
2  int var1;
3  }
```

Ahora, el compilador se dará cuenta de que **'ejemplo'** es un **'identificador'** y se le asigna el atributo 261, el cual es asignado a los **'identificadores'**, el compilador hace el match de $X = A$, es decir, $X = id$ y $A = ejemplo = id$:

```
-----  
Símbolo actual en X: id  
Token actual en A: ejemplo  
-----  
Contenido actual de la pila:  
id  
{  
lista_sent  
}  
$  
-----  
Tabla de símbolos  
-----  
LEXEMA      ATRIBUTO      CATEGORIA LÉXICA  
ejemplo      261             Identificadores  
-----
```

En la siguiente imagen, la **'Pila'** nos indica que **tiene que seguir una llave de apertura '{'**, y como es así, obtenemos el match de $X = A$:

```
-----  
Símbolo actual en X: {  
Token actual en A: {  
-----  
Contenido actual de la pila:  
{  
lista_sent  
}  
$  
-----  
Tabla de símbolos  
-----  
LEXEMA      ATRIBUTO      CATEGORIA LÉXICA  
ejemplo      261             Identificadores  
-----
```

Ahora:

lista_sent → sentencia sent_final

```
-----  
Símbolo actual en X: lista_sent ←  
Token actual en A: int  
-----  
Contenido actual de la pila:  
lista_sent ←  
}  
$  
-----  
Producción Actual de Derivación:  
2.-lista_sent ? sentencia    sent_final ←  
-----  
Tabla de símbolos  
-----  
LEXEMA                      ATRIBUTO                      CATEGORIA LÉXICA  
ejemplo                      261                      Identificadores  
-----
```

A continuación, se sustituirá

lista_sent en la 'Pila' por su producción: **sentencia** **sent_final**

```
-----  
Símbolo actual en X: sentencia ←  
Token actual en A: int  
-----  
Contenido actual de la pila:  
sentencia ←  
sent_final  
}  
$  
-----  
Producción Actual de Derivación:  
5.-sentencia ? tipo lista_id; ←  
-----  
Tabla de símbolos  
-----  
LEXEMA                      ATRIBUTO                      CATEGORIA LÉXICA  
ejemplo                      261                      Identificadores  
-----
```

Posteriormente **sentencia** se sustituye por la **producción** que ella misma generó:

sentencia → **tipo** **lista_id** ;

```
-----
Símbolo actual en X: tipo
Token actual en A: int
-----
Contenido actual de la pila:
tipo
lista_id
;
sent_final
}
$
-----
Producción Actual de Derivación:
22.-tipo ? int
-----
Tabla de símbolos
-----
LEXEMA      ATRIBUTO      CATEGORIA LÉXICA
ejemplo      261            Identificadores
-----
```

A lo largo de todo esto, como las **producciones han producido otras acciones/producciones**, no realizamos un “**check**” entre **X** y **A** aún.

Posteriormente, como se observa en la imagen de arriba, **tipo** produjo **int**

tipo → **int**

Y se sustituye en la estructura de datos ‘Pila’ como se muestra a continuación:

```
-----
Símbolo actual en X: int
Token actual en A: int
-----
Contenido actual de la pila:
int
lista_id
;
sent_final
}
$
-----
Tabla de símbolos
-----
LEXEMA      ATRIBUTO      CATEGORIA LÉXICA
ejemplo      261            Identificadores
-----
```

En la imagen anterior, podemos observar que **existe un match** entre **X** y **A**, es decir:

$X = \text{int}$ y $A = \text{int}$

Esto nos indica que el **código** escrito en **programa.txt** va **cumpliendo** con las **reglas** que impuso la **gramática**.

En la ejecución siguiente, podemos ver que es turno de **lista_id**

lista_id → **id** **id_final**

A su vez, agregamos otro **identificador** a nuestra tabla de símbolos, que en este caso es **'var1'**:

```
-----  
Símbolo actual en X: lista_id  
Token actual en A: var1  
-----  
Contenido actual de la pila:  
lista_id  
;  
sent_final  
}  
$  
-----  
Producción Actual de Derivación:  
9.-lista_id ? id id_final  
-----  
Tabla de símbolos  
-----  
LEXEMA      ATRIBUTO      CATEGORIA LÉXICA  
ejemplo      261            Identificadores  
var1          261            Identificadores  
-----
```

A continuación...

lista_id se sustituye por su producción **id id_final** en el contenido actual de la **'Pila'**

Y volvemos a tener un match entre **X** y **A**:

Es decir, $X = id$ y $A = var1 = id$

```
-----  
Símbolo actual en X: id  
Token actual en A: var1  
-----  
Contenido actual de la pila:  
id  
id_final  
;  
sent_final  
}  
$  
-----  
Tabla de símbolos  
-----  
LEXEMA      ATRIBUTO      CATEGORIA LÉXICA  
ejemplo     261           Identificadores  
var1        261           Identificadores  
-----
```

Continuamos...

Una de las producciones de **id_final** (que es el **próximo ítem** indicado por el contenido de la 'Pila') es:

id_final → **E**

Como lo muestra la gramática en las líneas 10 y 11:

10.- id_final	→ , id id_final
11.- id_final	→ ϵ

```
-----
Símbolo actual en X: id_final
Token actual en A: ;
-----
Contenido actual de la pila:
id_final
;
sent_final
}
$
-----
Producción Actual de Derivación:
11.-id_final ?  $\epsilon$ 
-----
Tabla de símbolos
-----
LEXEMA      ATRIBUTO      CATEGORIA LÉXICA
ejemplo      261            Identificadores
var1         261            Identificadores
-----
```

Con ayuda de la **matriz predictiva** y el código escrito en **programa.txt**, el algoritmo decide producir la **acción** de la **línea número 11.-** de la **gramática**, la cual indica **vacío**.

La razón por la que existen estos **dos tipos de producciones** en **id_final** es porque se pudieran tener **más identificadores escritos** en el código de **programa.txt**, y eso está absolutamente permitido por esta gramática.

Por ejemplo, ambas de las siguientes líneas son correctamente léxica y sintácticamente:

Opción 1 (Con la que estamos **trabajando actualmente**):

```
programa.txt X
src > archivos > programa.txt
1 class ejemplo {
2 int var1;
3 }
```

Opción 2 (Si ejecutamos este programa también **deberá ser correcto**):

```
programa.txt X
src > archivos > programa.txt
1 class ejemplo {
2 int var1, var2, var3;
3 }
```

Ahora continuamos...

Después de **E (vacío)**, analizamos el **ítem ‘;’** indicado por la **‘Pila’** y obtenemos **match** entre X y A, es decir:

X=; y **A=;**

```
-----
Símbolo actual en X: ;
Token actual en A: ;
-----
Contenido actual de la pila:
;
sent_final
}
$
-----
Tabla de símbolos
-----
LEXEMA      ATRIBUTO      CATEGORIA LÉXICA
ejemplo     261            Identificadores
var1        261            Identificadores
-----
```

Volvemos a encontrar vacío aquí:

```
-----
Símbolo actual en X: sent_final
Token actual en A: }
-----
Contenido actual de la pila:
sent_final
}
$
-----
Producción Actual de Derivación:
4.-sent_final ? ε
-----
Tabla de símbolos
-----
LEXEMA      ATRIBUTO      CATEGORIA LÉXICA
ejemplo      261            Identificadores
var1         261            Identificadores
-----
```

Ya que de acorde a la gramática en la **línea 3** podríamos tener lo siguiente y también es correctamente léxico y sintácticamente:

3.- sent_final	→ sentencia sent_final
4.- sent_final	→ ε

Ejemplo de cómo se vería la **línea 3.-** en el código de **programa.txt** (Correcto):

```
≡ programa.txt X
src > archivos > ≡ programa.txt
1  class ejemplo {
2  int var1; float decimal;
3  }
```

Pero nosotros tenemos lo siguiente actualmente (**vacío**), que como se muestra a continuación, se puede observar que hay un '**vacío**' entre ';' y '}':

```
≡ programa.txt X
src > archivos > ≡ programa.txt
1  class ejemplo {
2  int var1; 
3  }
```

Seguimos...

Ahora es turno de analizar nuestro **último ítem '{'**, que desde un principio, se encontraba hasta abajo en nuestra **estructura de datos 'Pila'**:

Obtenemos match en X y A, es decir:

X='}' y **A='}'**

```
-----
Símbolo actual en X: }
Token actual en A: }
-----
Contenido actual de la pila:
}
$
-----
Tabla de símbolos
-----
LEXEMA      ATRIBUTO      CATEGORIA LÉXICA
ejemplo      261            Identificadores
var1         261            Identificadores
-----
```

1. Llegamos al final de la **gramática** en **X** y al final de la lectura del código de **programa.txt** en **A**.
2. Se hace un último match entre **\$** en **X** y en **A**
3. La gramática no tiene ninguna instrucción más, por lo que obtenemos **null**.
4. El programa.txt es léxico y sintácticamente correcto

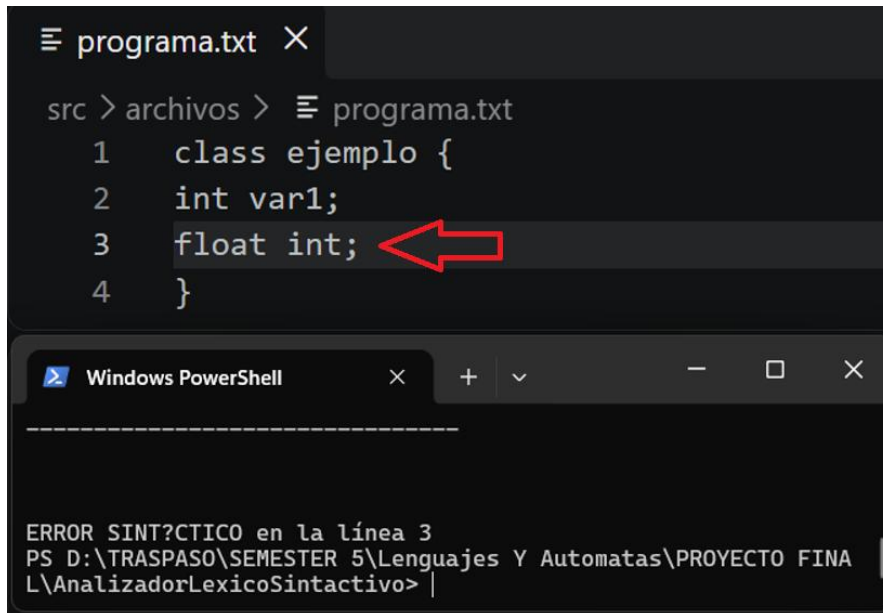
```
-----
Símbolo actual en X: $
Token actual en A: $
-----
Contenido actual de la pila:
$
-----
Tabla de símbolos
-----
LEXEMA      ATRIBUTO      CATEGORIA LÉXICA
ejemplo      261            Identificadores
var1         261            Identificadores
-----

Símbolo actual en X: null
Token actual en A: $
-----
Contenido actual de la pila:
-----
Tabla de símbolos
-----
LEXEMA      ATRIBUTO      CATEGORIA LÉXICA
ejemplo      261            Identificadores
var1         261            Identificadores
-----

LA PILA EST? VAC?A
EL PROGRAMA ES LÉXICA Y SINT?CTICAMENTE CORRECTO
PS D:\TRASPASO\SEMESTER 5\Lenguajes Y Automatas\PROYECTO FINAL\AnalizadorLexicoSintactivo> |
```

Resultados de **otras ejecuciones** del proyecto al correr con **diferentes códigos** escritos en programa.txt:

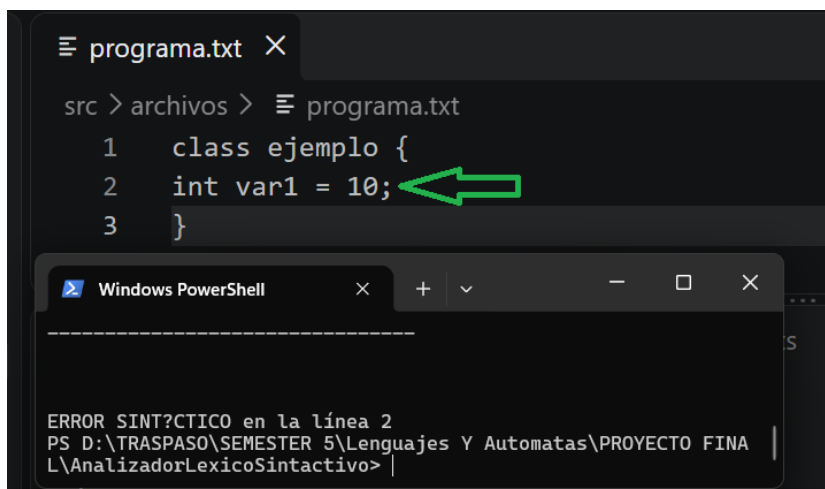
1. **Error** por intentar nombrar como **'int'** (palabra reservada del lenguaje) la **variable float**:



```
programa.txt X
src > archivos > programa.txt
1  class ejemplo {
2  int var1;
3  float int;
4  }
```

```
Windows PowerShell
-----
ERROR SINT?CTICO en la línea 3
PS D:\TRASPASO\SEMESTER 5\Lenguajes Y Automatas\PROYECTO FINA
L\AnalizadorLexicoSintactivo>
```

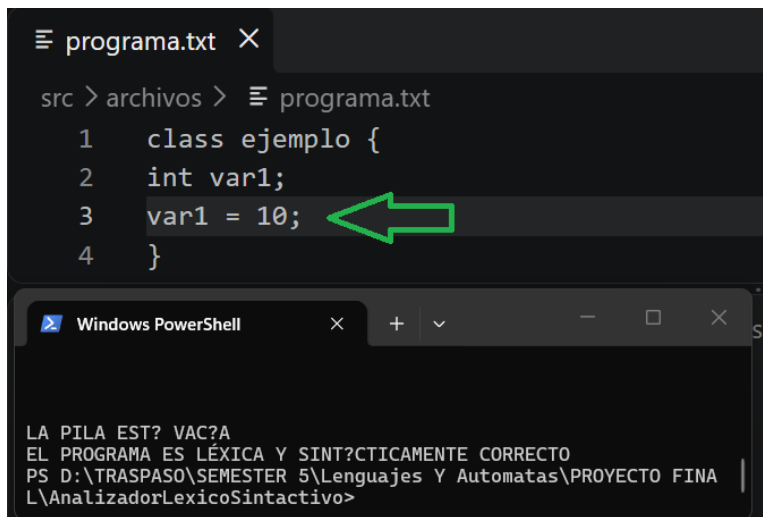
2. **Declaración e inicialización directa no permitida** por la gramática:



```
programa.txt X
src > archivos > programa.txt
1  class ejemplo {
2  int var1 = 10;
3  }
```

```
Windows PowerShell
-----
ERROR SINT?CTICO en la línea 2
PS D:\TRASPASO\SEMESTER 5\Lenguajes Y Automatas\PROYECTO FINA
L\AnalizadorLexicoSintactivo>
```

3. Inicialización sin declaración **Si** permitida:

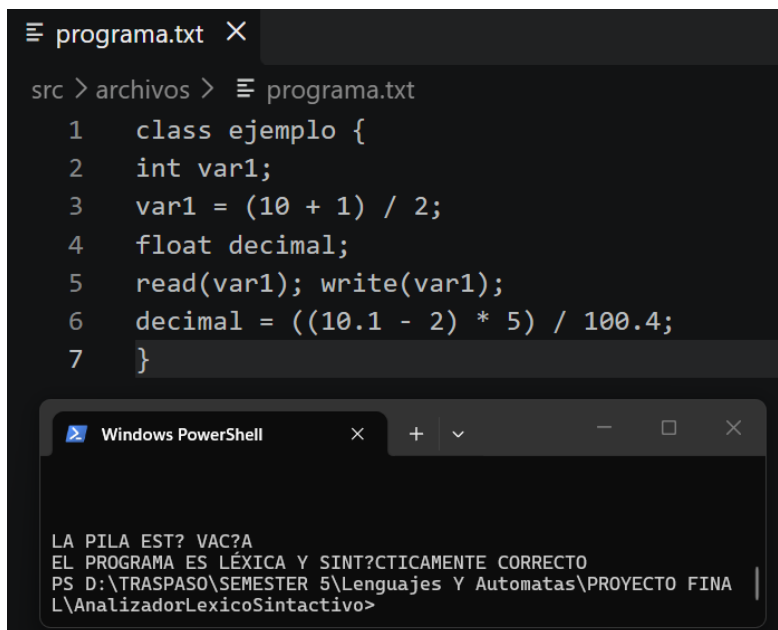


```
src > archivos > programa.txt
1  class ejemplo {
2  int var1;
3  var1 = 10;
4  }
```

Windows PowerShell

LA PILA EST? VAC?A
EL PROGRAMA ES LÉXICA Y SINT?CTICAMENTE CORRECTO
PS D:\TRASPASO\SEMESTER 5\Lenguajes Y Automatas\PROYECTO FINA
L\AnalizadorLexicoSintactivo>

4. Código escrito léxica y sintácticamente **correcto**, cumpliendo con reglas impuestas por la gramática:



```
src > archivos > programa.txt
1  class ejemplo {
2  int var1;
3  var1 = (10 + 1) / 2;
4  float decimal;
5  read(var1); write(var1);
6  decimal = ((10.1 - 2) * 5) / 100.4;
7  }
```

Windows PowerShell

LA PILA EST? VAC?A
EL PROGRAMA ES LÉXICA Y SINT?CTICAMENTE CORRECTO
PS D:\TRASPASO\SEMESTER 5\Lenguajes Y Automatas\PROYECTO FINA
L\AnalizadorLexicoSintactivo>